

OpenAI API function calling 기능 실습

본 실습에서는 OpenAI Chat Completions API의 **함수 호출(Function Calling)** 기능을 사용하여, 모델이 외부 함수를 호출하고 그 결과를 응답에 활용하는 방법을 단계별로 살펴보겠습니다.

1. 함수 호출 기능 소개

함수 호출 기능은 모델이 사전에 정의된 함수를 필요에 따라 실행하도록 도와주는 강력한 도구입니다. 이 기능을 통해 어시스턴트는 자체 지식에 없는 **실시간 정보 조회**나 **복잡한 계산 작업** 등을 외부 함수에 위임할 수 있습니다. 예를 들어 사용자가 날씨를 물어보면, 어시스턴트는 날씨 API와 연결된 함수를 호출하여 최신 기온을 가져온 뒤 응답할 수 있습니다. 이처럼 함수 호출을 활용하면:

- **실시간/외부 데이터 활용:** 모델이 최신 정보(날씨, 주가, 뉴스 등)를 함수로부터 받아와 답변에 반영할 수 있습니다.
- **모델 한계 보완:** 수학 계산, 데이터베이스 질의 등 모델이 직접 처리하기 어려운 요청을 외부 로직으로 해결할 수 있습니다.
- **도메인 확장:** 개발자가 정의한 임의의 함수로 모델의 기능을 확장할 수 있어, 특정 분야나 서비스에 특화된 어시스턴트를 구현할 수 있습니다.

현재 제공되는 대부분의 OpenAI 모델(gpt-4.1 계열, gpt-4o 계열 등)은 이 함수 호출 기능을 지원합니다. 모델에게 함수 목록을 제공하면, 질문 의도에 따라 적절한 함수를 선택해 필요한 인수를 함께 호출 형식으로 응답을 반환합니다. 이제 이러한 함수 호출을 구현하는 방법을 예제로 알아보겠습니다.

2. API 키 설정 및 OpenAI 클라이언트 초기화

OpenAI API를 사용하기 위해 먼저 API 키를 준비해야 합니다. API 키는 OpenAI 계정의 대시보드에서 생성할 수 있으며, 노출되지 않도록 환경 변수 등에 저장하여 사용합니다. 본 예제에서는 **python-dotenv**를 이용해 `.env` 파일에 저장된 키를 로드하고, OpenAI Python SDK의 `OpenAI` 클라이언트를 초기화합니다.

아래 코드에서는 `.env` 파일에서 키를 읽어와 `client = OpenAI()`로 클라이언트를 생성합니다. (API 키가 올바르게 설정되어 있으면 `OpenAI()` 생성자에서 자동으로 키를 불러옵니다.)

3. 외부 함수 정의 및 테스트

이제 함수 호출 기능을 체험하기 위한 예시로 **날씨 정보를 가져오는 함수**를 만들어보겠습니다. 사용자가 "서울 날씨 어때요?"라고 물어보면, 어시스턴트가 이 함수를 호출하여 실시간 날씨 정보를 얻어 답변하도록 해볼 것입니다.

예를 위해 간단한 `get\_weather` 함수를 정의하겠습니다. 이 함수는 위도(latitude)와 경도(longitude)를 받아 해당 위치의 현재 기온을 섭씨로 반환합니다. 구현에는 오픈 메테오(Open-Meteo)라는 공개 기상 API를 사용하여, 주어진 좌표의 현재 기온 데이터를 가져옵니다. 함수는 `requests` 라이브러리를 통해 API를 호출하고, 응답 JSON에서 온도 값을 추출하여 반환합니다.

함수를 정의한 후, 예시 좌표에 대해 함수를 호출해 제대로 동작하는지 확인해 보겠습니다. 서울의 대략적인 좌표(위도 37.484642, 경도 126.930063)를 입력으로 주었을 때 온도가 잘 반환되는지 출력해 보겠습니다.

```
def get_weather(latitude=37.484859, longitude=126.930086):  
  
    url = f"https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current=temperature_2m"  
  
    print(url)
```

4. 함수 툴 설정 및 모델 응답 생성

이제 OpenAI API에 우리가 정의한 함수를 **툴(tool)**로 등록하고, 모델이 이 함수를 호출하도록 유도해 보겠습니다.

이를 위해 먼저 `tools`라는 리스트에 함수 정보를 구성합니다. 각 함수 툴에는 이름(name), 설명

(description), 그리고 매개변수(parameters) 스키마를 지정해야 합니다. 아래 코드에서는 `type`: "function"으로 함수를 정의하고, `name`: "get_weather", `description`: "저장된 좌표의 현재 온도를 섭씨 단위로 구합니다"와 같이 설정했습니다. 또한 `parameters` 필드에 함수가 요구하는 인자들의 JSON 스키마(여기서는 latitude와 longitude 숫자형, 둘 다 필수)를 명시했습니다.

그 다음 어시스턴트에게 보낼 `messages`를 준비합니다. 사용자의 질문으로 "오늘 서울 날씨 어때요?"라는 메시지를 추가하겠습니다. 이제 이 사용자 메시지와 함께 `client.chat.completions.create`를 호출하여, 모델의 응답을 받아보겠습니다. `tools` 파라미터에 앞서 정의한 함수 정보를 포함시켰으므로, 모델은 답변 과정에서 이 함수를 호출할 수 있게 됩니다.

모델이 함수 호출을 결정하면, 응답으로 일반 텍스트 대신 ****함수 호출 요청****을 반환하게 됩니다. 즉, 어시스턴트는 직접 답을 주는 대신 `get\_weather` 함수를 특정 인수로 호출하라는 정보를 주게 됩니다.

아래 코드에서는 ChatCompletion API를 호출하여 이러한 과정을 실행하고, 결과를 `completion` 변수에 저장합니다.

5. 함수 호출 결과 처리 및 최종 답변

모델이 함수 호출 요청을 반환했다면, 이제 개발자 측에서 해당 함수를 실제로 실행하고 그 결과를 어시스턴트에게 전달해야 합니다.

우선 `completion.choices[0].message.tool\_calls`를 통해 모델이 요청한 함수 호출 정보를 확인해보겠습니다. 여기에는 호출하려는 함수 이름과 전달된 인자들이 포함되어 있습니다. 우리 예시의 경우 모델은 `get\_weather` 함수를 호출하도록 응답했을 것이며, 서울의 위도와 경도에 해당하는 값을 인자로 제공했을 것입니다.

함수 실행 결과를 어시스턴트 대화에 추가해 보겠습니다. `messages.append(...)`를 사용하여, 먼저 어시스턴트의 함수 호출 메시지를 대화 내역에 넣고, 이어서 'tool' 역할의 메시지를 추가합니다. 이 'tool' 메시지에는 함수 호출의 `id`와 실행 결과를 문자열로 담았습니다. 이렇게 하면 OpenAI API는 해당 함수 호출에 대한 결과를 받았다고 인식하게 됩니다.

이제 `messages` 리스트를 출력해서 대화 내역에 함수 호출 결과까지 올바르게 들어갔는지 확인해 봅시다. 사용자 메시지, 어시스턴트의 함수 호출 메시지, 그리고 툴의 응답 메시지가 순서대로 포함되어 있을 것입니다.

이제 준비된 대화 내역을 바탕으로 다시 한 번 모델에게 최종 답변을 요청해보겠습니다. ``messages``에는 사용자 질문과 그에 대한 함수 호출 및 결과가 모두 포함되어 있으므로, 모델은 이 정보를 활용해 사용자에게 답변할 것입니다.

주의할 점은, **함수 호출 응답 뒤에는 반드시 해당 함수 호출에 대한 `'tool'` 메시지가 존재해야 한다는 것**입니다. 만약 모델이 여러 개의 함수를 동시에 호출했는데 그중 일부에 대한 결과 메시지가 누락되면, "처리되지 않은 `tool_call_id`가 있다"는 오류가 발생합니다.

우리 예시는 ``tools`` 호출 시 ``parallel_tool_calls=False``로 지정해 두었기 때문에, 모델이 함수를 한 번에 하나씩만 호출합니다. 그래서 ``tool_call`` 하나에 결과 메시지 하나만 짝지어 넣어도 누락이 생기지 않고, 아래 요청은 정상적으로 최종 답변을 받아옵니다. (만약 ``parallel_tool_calls``를 켜서 여러 함수가 동시에 호출되는 경우라면, 모든 ``tool_call``에 대해 각각 결과 메시지를 추가해야 오류 없이 진행됩니다.)